

Reconocimiento facial personal en Matlab

Hernán Capilla Urbano

Abstract—Software de detección de rostros en imágenes a raíz de un entrenamiento previo con una numerosa batería de imágenes del sujeto en cuestión a detectar.

Una vez realizado dicho entrenamiento, el software estará preparado para abordar la detección del rostro del sujeto, basado en los datos entrenados, con nuevas y distintas imágenes.

El software es capaz de discriminar entre rostros diferentes, detectando y señalando en la imagen únicamente el del sujeto en cuestión a analizar.

Keywords—Face detection, train & test, cascadeObjectDetector, computer vision.



1 INTRODUCCIÓN

Este proyecto consiste en el reconocimiento facial. Más concretamente el de mi propia cara. Crear un programa o un software que se fije y almacene mis rasgos faciales en base a una batería de imágenes propias preseleccionadas y que a través de estas pueda llegar a realizar una detección de mi cara (únicamente de ésta) en una imagen.

En este proyecto también se contempla el hecho de que el programa desarrollado discrimine mi cara respecto a otras en una imagen en la que saliesen varias personas incluyéndome a mí.

Así pues y como he comentado, la detección sería personal, y se planteará todo el problema dando por sentado que el objetivo principal es la detección de únicamente mi cara en diferentes imágenes a través del entrenamiento o del almacenamiento de los rasgos que componen mi estructura facial.

El entrenamiento será llevado a cabo en base a una batería de imágenes propias en las que se hallará mi cara en diferentes posturas y realizando diferentes gestos sobre un fondo con varios elementos. Buscando así potenciar el factor discriminante de la aplicación y no únicamente detectar mi cara sobre fondo plano.

Pueden existir varias aplicaciones para este programa que desarrollo, como puede ser vigilancia de una persona en concreto. Entrenar al programa con muchas imágenes en diferentes perspectivas de una persona en concreto para que después se encargue automáticamente de la detección de la misma en imágenes diferentes en las que no se ha entrenado. Como pueden ser imágenes de eventos, frames de una cámara de vigilancia, etc.

Otra aplicación al software desarrollado estaría más orientada a un juego por decirlo de alguna manera. Todos conocemos los libros de "Encuentra a Wally", se podría emplear el programa para entrenarlo con muchas imágenes de Wally y que después él se encargase en los nuevos libros e imágenes de encontrar a Wally de forma automática.

Las aplicaciones a este software son muy concretas, pero no por ello menos interesantes. Podría llegar a ser modificado y mejorado para detectar a varias personas a la vez, incluso detectándolas de forma simultánea en una misma imagen. Por motivos de tiempo y de no ser el objetivo principal de este proyecto no se ha realizado esto, pero el código es muy amoldable e integrable para una posible aplicación del caso que acabo de comentar.

2 ESTADO DEL ARTE

2.1 CascadeObjectDetector para detectar bicicletas

[1] En este primer caso, se emplea el sistema de CascadeObjectDetector para localizar bicicletas situadas en diferentes puntos de una ciudad (ver Fig. 1).

Se basa en un batch inicial de unas 350 imágenes de bicicletas y emplea estas imágenes para entrenar a Cascade y que éste aprenda a diferenciar a las bicicletas del escenario y localizarlas.

En este caso se barajan y explican muchos conceptos a cerca de CascadeObjectDetector (detector que yo mismo usaré durante este proyecto) como las regiones de interés (ROI) y el False Alarm Rate (FAR) que son muy útiles a la hora de entrenar un modelo.



Fig. 1. Escena donde se halla una bicicleta ya localizada por Cascade después de ser entrenado con imágenes anteriores de bicicletas.

2.2 Algoritmo KLT para detección y tracking de caras en tiempo real

[2] Este segundo caso está orientado al reconocimiento de caras en tiempo real, como puede ser a través de un vídeo o de una webcam (ver Fig. 2).

Este caso es interesante puesto que se basa en el algoritmo KLT y a su vez emplea el sistema CascadeObjectDetector para detectar la cara de una persona, almacenar los rasgos dentro de la región detectada como cara, y emplearlos para realizar un seguimiento en los diferentes frames de un vídeo o que puedan provenir de una webcam para ir señalizando la cara en tiempo real.

Aporta muchas ideas nuevas y junto con el primer caso dan mucho juego a realizar el proyecto que quiero desarrollar. Explora bien los términos y pasos a seguir y el por qué de cada paso.

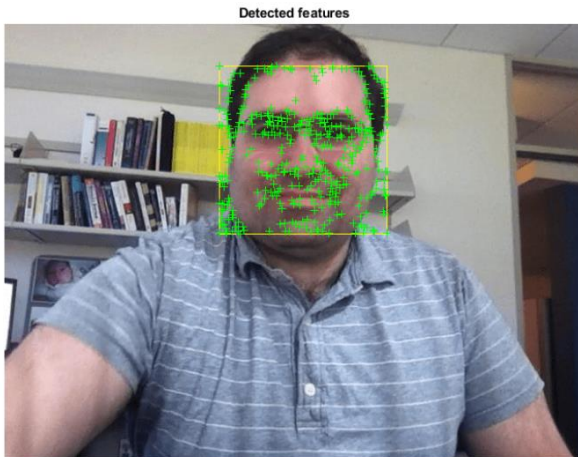


Fig. 2. Imagen tomada en el momento en que se ha aplicado el CascadeObjectDetector y se ha hecho un seguimiento de sus rasgos faciales, siendo estos indicados por pantalla en tiempo real.

3 PROPUESTA

3.1 Selección del sistema basado en los casos vistos

En este apartado explicaré cómo he abordado el problema, el proceso seguido y en qué me he basado.

Primera mente, me he basado mucho en lo aprendido en los casos que he presentado en el apartado de Estado del arte. Dichos proyectos usan CascadeObjectDetector, uno para detección de objetos y el otro para detectar el rostro de una persona. Es debido a esto que supe en un primer lugar que debía seguir este sistema (COD) para poder llegar a detectar mi propio rostro.

En el primer caso, es muy interesante el concepto del uso que le da a COD, puesto que no lo emplea de la forma habitual, consistente en que aprenda de una sola imagen; sino que lo entrena desde un principio con un set de 350 imágenes en las cuales en cada una había en algún punto u otro una bicicleta. Además de esto, le aportaba al sistema COD otro set de imágenes en las cuales habían entornos muy parecidos a los aportados con anterioridad pero esta vez sin bicicletas, de forma que el COD pudiese discernir cuándo había y cuándo no, el objeto a detectar.

Esto me ha dado ideas de cómo enfocar mi proyecto, necesitaré un batch de imágenes, tanto positivas (con la persona a identificar) como negativas (sin la persona pero fondo semejante). Además de esto, un muestrario de fotografías objetivo en las que probar el modelo COD entrenado y comprobar que efectivamente los resultados son los esperados.

Otro paso importante es cómo entrenar a este COD, en el primer caso también se explicaba y daban ideas de cómo hacerlo. Existen diversos métodos para ello, es bastante flexible para esto dentro de lo que cabe, pero yo me decanto por emplear la aplicación de Matlab de Image Labeler (anteriormente se llamaba Training Image Labeler). Con esta aplicación del propio Matlab, se pueden seleccionar las Regiones De Interés (ROI, en inglés) de cada una de las imágenes con las que se entrenará COD y serán aquellas en las que se fijará el sistema para encontrar similitudes con el objeto a detectar.

3.2 Exploración del sistema y funcionamiento de CascadeObjectDetector (COD)

Una vez teniendo claro el sistema a emplear, que será CascadeObjectDetector, es imperante saber cómo funciona, en qué se basa y qué puede aportarnos.

El CascadeObjectDetector, o COD para abreviar, es un detector que se puede emplear para detectar tanto caras, narices, bocas, cuerpos, etc. Y no solo se queda ahí, sino que también puede ser empleado para detectar coches o señales de tráfico o, como hemos visto en el caso 1, bicicletas. En resumen, es un sistema muy flexible para detectar todo tipo de objetos y formas cuyo ratio de aspecto no varíe significativamente (a pesar de que con las imágenes de entrenamiento necesarias se podría ajustar esto y que el impacto fuese menor).

Hasta el momento sabemos que sirve para detectar objetos, pero... ¿cómo lo hace? [3]

Pues bien, lo que hace COD es identificar los diferentes objetivos mediante ir pasando por la imagen a detectar un plano de aquellos objetos con los que él ha entrenado para ver si las facciones que tiene almacenadas del entrenamiento corresponden o están correlacionadas con la imagen que tiene que analizar.

Pero para realizar esto es necesario un entrenamiento previo con muchas imágenes para que COD tenga una base de datos muy grande entre la que comparar, cuantas más imágenes, mejor, puesto que tendrá más con lo que comparar, con lo cual se cometerán menos fallos.

En el entrenamiento, un modelo de COD funciona por diferentes stages. En cada stage se realiza una localización de objetos en base a lo que sabe, va deslizando las imágenes del objeto a detectar con lo que sabe de base y va comprobando si da positivo (detectado) o negativo (no detectado). Si el resultado es negativo, comprende que ahí no hay nada más que buscar y pasa a la siguiente región de la imagen que está detectando. Si fuese positiva, se da por hecho que ha detectado al objeto y pasa al siguiente stage (ver Fig. 3).

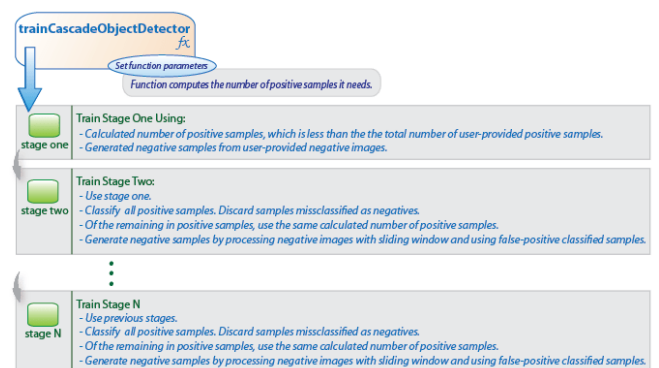


Fig. 3. Secuencia del COD stage por stage con los detalles de qué ocurre en cada stage del sistema.

A cada stage se aplica una política de Boosting, consistente en dar pesos a las entradas y dar más valor a aquellos que no han sido clasificados correctamente, de esta forma el modelo puede aprender de sus errores y solucionarlos en un futuro o evitar que vuelvan a suceder.

Otro concepto importante es el False Alarm Rate (FAR). Consiste en un parámetro que indica al modelo cuán grado de fallo puede permitirse o no en cuanto a los falsos positivos.

Para funcionar correctamente, dicho por la propia documentación de Matlab, es necesario un FAR pequeño, puesto que cuanto más pequeño es, más puede aprender el modelo de sus errores. Esto es así porque si un stage clasifica un objeto de forma negativa la clasificación para, y no puedes corregir el objeto, puesto que considera que en esa imagen no hay nada que corregir y por lo tanto no pasa al siguiente stage. Pero si el FAR es pequeño, puede permitirse “equivocarse” y en el siguiente stage tendrá más peso debido al Boosting y podrá ser solucionado el falso positivo.

En resumen, CascadeObjectDetector será el sistema escogido para el proyecto, y necesitaremos entrenarlo con diferentes y numerosas imágenes tanto positivas como negativas de mí en las que se me vea el rostro, pues ese es el objetivo, que sea capaz de detectarme. Una vez entrenado, probar el modelo en diferentes imágenes en las que yo no esté solo, para probar que pueda diferenciarme de las demás personas.

4 EXPERIMENTOS, RESULTADOS I ANÁLISIS

4.1 Obtención de los datos de entrenamiento

Seguiremos los pasos que hemos comentado en el apartado 3.2.

Lo primero a realizar será la obtención de diferentes baterías de imágenes. Por un lado, 50 imágenes positivas (ver Fig. 4a), que contendrán una muestra del objeto a identificar (es decir, mi cara), y servirán de entrenamiento para el modelo COD para que sepa qué es lo que busca; en contraposición, 18 imágenes negativas (ver Fig. 4b), que no contendrán ningún ejemplar de él (no saldré en la imagen); y 6 imágenes objetivo, que serán aquellas imágenes que no se encuentran en ninguna de las baterías anteriormente mencionadas y que servirán como test para comprobar que el modelo funciona de forma adecuada.



Fig. 4a) (izquierda) y 4b) (derecha). Imágenes dentro de la batería de positivas (4a) y negativas (4b).

Estas imágenes objetivas, como he comentado, no se encuentran en las baterías anteriores, pese a que muestran similitudes (tanto en entorno como en posición de mi cuerpo y cara) con las imágenes que en ellas se encuentran. A pesar de esto, hay un par de imágenes objetivo cuyo escenario ha cambiado completamente de las positivas (ver Fig. 5b) y un parque, a pesar de estar en el mismo entorno, hay situadas dos personas

en vez de únicamente yo, para comprobar que el modelo efectivamente es capaz de discriminar entre varias personas detectándose únicamente a mí (ver Fig. 5a).



Fig. 5a) (izquierda) y 5b) (derecha). Imágenes dentro de la batería de objetivo. Por un lado (5a) dos personas y únicamente mi rostro a detectar y en el otro lado (5b) cambio de escenario para comprobar que no se haya producido overfitting en el modelo a raíz del escenario.

Las imágenes han sido tomadas con un iPhone 12 y exceptuando la de la Figura 5b todas han sido situadas en ese mismo escenario: un parque.

4.2 Creación y entrenamiento del modelo

Para la creación del modelo primero hemos tenido que seleccionar manualmente las Regiones De Interés (ROI) en las que se fijará el modelo con la App Image Labeler de Matlab (ver Fig. 6).

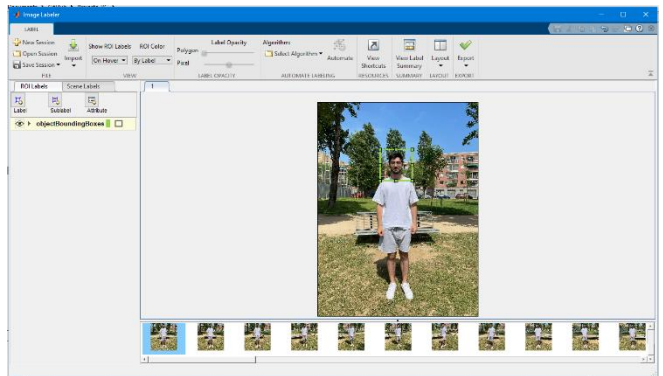


Fig. 6. Interfaz de la App Image Labeler de Matlab, en ésta podemos ver cómo seleccionamos manualmente las ROI para cada una de las imágenes dentro de la batería.

Una vez seleccionadas, son exportadas como struct porque sinó la función CascadeObjectDetector de Matlab no los puede procesar. En este caso me he encontrado con la problemática de que la documentación que he mirado es de hace años de cuando esta aplicación se llamaba Training Image Labeler y sí permitía exportar en formato struct, ahora no lo permite. Teniendo un formato de 2xN, siendo el primer campo el path de la imagen, y el segundo las 4 coordenadas de la ROI, formada por un cua-

drado. La N sin embargo, será el número de imágenes que contiene la batería de positivas (ver Fig. 7).

Únicamente tendremos el struct de positivas, puesto que estas a diferencia de las negativas tienen ROI. Las negativas al no contener el objeto no tienen estas regiones y, por lo tanto, no necesitan el struct sino únicamente el directorio en el que se encuentran.

positivas			
1x50 struct with 2 fields			
Fields	imageFilename	objectBoundingBoxes	
1	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\1.jpg'	[639,469,224,293]	
2	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\2.jpg'	[648,448,212,320]	
3	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\3.jpg'	[654,427,236,320]	
4	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\4.jpg'	[669,451,218,332]	
5	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\5.jpg'	[666,436,224,308]	
6	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\6.jpg'	[684,472,218,299]	
7	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\7.jpg'	[511,382,271,380]	
8	'C:\Users\98cap\Documents\GitHub\Projecte-VC\positivo\8.jpg'	[591,388,260,365]	

Fig. 7. Formato del struct que introduciremos en el COD, formato por el path de la imagen (imageFilename) y las coordenadas de la ROI (objectBoundingBoxes).

Tal y como he comentado con anterioridad, la nueva versión de la Image Labeler App no permite exportar en formato de struct de 2xN sino como listas. Esto no sería un problema si la función CascadeObjectDetector acogiese los parámetros en formato de lista, pero no es el caso, necesita un struct. Con lo cual la solución que encontré a esto fue crear un struct de zero con los campos en el formato correcto e ir copiando uno por uno los valores dentro de cada celda. No encontré ninguna otra solución a esto.

Una vez tenemos el struct de positivas, hemos de crear y entrenar el modelo.

Para esto le pasaremos por parámetro a la función CascadeObjectDetector tanto el struct que acabamos de crear, como el path del directorio donde tenemos las imágenes negativas.

Además de estos, le pasaremos el FAR y otra variable para indicar el número de Stages que realizará el modelo. Recordemos que estos dos conceptos habían sido explicados en el 3.2.

Una vez indicado todos estos parámetros, el COD se pondrá a entrenar con los datos que le han sido dados, detectando los objetos dentro de las ROI y aprendiendo de ello (ver Fig. 8).

```
Used 50 positive and 48 negative samples
Time to train stage 4: 19 seconds

Training stage 5 of 10
[.....Warning:
Unable to generate a sufficient number of negative samples for this stage.
Consider reducing the number of stages, reducing the false alarm rate
or adding more negative images.

Cannot find enough samples for training.
Training will halt and return cascade detector with 4 stages
Time to train stage 5: 17 seconds

Training complete
```

Fig. 8. Consola una vez ha terminado de entrenarse el COD. Este modelo en concreto tenía un nStages=10 y un FAR=0.000001.

Un problema que podemos hallar a la hora de entrenar al COD es que los nStages que pretendemos que realice sean demasiados. Puede suceder que no tenga datos suficientes de

entrada como para no llegar a realizar tantos Stages y pareantes. Tal y como es el caso mostrado en la Figura 8. No ha llegado a realizar 10 stages que era lo que yo le había indicado que hiciese y únicamente ha llegado a realizar 5.

El entrenamiento del COD habrá finalizado una vez realizados los pasos vistos hasta el momento. Ahora observemos los resultados obtenidos y por último, los resultados considerados como aceptables o esperados.

4.3 Experimentos

Han sido necesarias muchas pruebas para obtener unos resultados que se puedan siquiera mostrar, puesto que muchas veces no detectaba las caras.

Voy a mostrar algunos ejemplos de lo que ocurre cuando ponemos un FAR demasiado alto y un nStages bajo (ver Fig. 9a y 9b).



Fig. 9a) (izquierda) y 9b) (derecha). En el 9a), resultado de nStages=4 y FAR=30. En el 9b), resultado de nStages=4 y FAR=50.

Podemos observar en la Figura 9 como con un FAR muy alto puede provocar muchos falsos positivos (esto lo habíamos explicado ya) y por lo tanto provoca que el programa falle, detectando casi cualquier cosa.

Solucionamos esto dándole un valor más reducido al FAR y en cuanto a nStages tenemos dos opciones, por un lado, ir aumentando poco a poco y ver si llega a más Stages con los inputs que tenemos, o dejar la opción por defecto (que son 20 stages) y que Matlab llegue hasta donde pueda. En mi caso vinecesaria la segunda opción para llegar a los parámetros que mejores resultados me han dado, llegando hasta 7 stages.

4.4 Resultados

Hablemos ahora de los mejores resultados que he obtenido y de lo que he considerado como el resultado deseado del proyecto. Todos estos resultados que veréis a continuación han sido obtenidos con nStages=7 y FAR=0.000001 (os dije que cuanto menor el FAR, mejor).

Los resultados de las detecciones que he obtenido vienen en el orden en que los comento ahora mismo: fotografía de cerca, fotografía de lejos, fotografía con otra persona, fotografía en otro escenario.

Veremos el primer y segundo caso: Fotografía de cerca (ver Fig. 10a) y Fotografía de lejos (ver Fig. 10b).



Fig. 10a) (izquierda) y 10b) (derecha). Imágenes resultado de la detección una vez aplicado el modelo comentado al inicio de este apartado. A la izquierda (10a) fotografía de cerca, y a la derecha (10b) fotografía de lejos.

Observamos que el resultado es muy distinto a los experimentos vistos con anterioridad. Detecta mi cara perfectamente tanto cerca como lejos.

Vayamos ahora a la fotografía que demuestra si realmente mi modelo discrimina entre varias personas (ver Fig. 11). Esta fotografía es muy importante puesto que demuestra un correcto funcionamiento de este y que verdaderamente se haya quedado con mis rasgos y facciones.



Fig. 11. Imagen resultado de la detección con dos personas distintas en la que podemos observar cómo el modelo creado únicamente señala mi cara, demostrando así un correcto funcionamiento.

Nuestro objetivo ha sido cumplido, pero antes de acabar lo pondremos a prueba una última vez para ver si en un cambio de escenario sigue actuando igual de bien (ver Fig. 12).



Fig. 12. Imagen resultado de la detección en la que ha habido un cambio de escenario respecto al parque. Se puede observar que la detección se ha realizado correctamente.

Vemos que el resultado es satisfactorio, ante unos rasgos y pose parecidas, funciona.

5 CONCLUSIONES

Las conclusiones extraídas de este proyecto son que los resultados de un modelo entrenado tan a pequeña escala son más satisfactorios. Digo a pequeña escala porque la batería de imágenes consistía únicamente en 50 positivas y 18 negativas. Con cientos de imágenes habrían sido mucho mejores los resultados y el modelo sería muchísimo más flexible para diferentes escenarios.

También es importante que hayan más imágenes por el hecho de que el modelo no era capaz de llegar a entrenarse más allá de las 7 stages, que es lo máximo que he conseguido. Esto por falta de imágenes con las que contrastar.

En general pero, estoy muy contento con los resultados y con haber aprendido el funcionamiento de la detección facial y de cómo sistemas como CascadeObjectDetector se nutren y entrenan de las imágenes para almacenar y quedarse con los rasgos de las personas u objetos a detectar.

BIBLIOGRAFIA

- [1] Canal de Matlab oficial de YouTube. “Object Detection: Part 1 | Student Competition: Computer Vision Training.”
- [2] Documentación de Matlab oficial de la página web oficial. “Face Detection and Tracking Using the KLT Algorithm”.
- [3] Documentación de Matlab oficial de la página web oficial. “Get Started with Cascade Object Detector”.